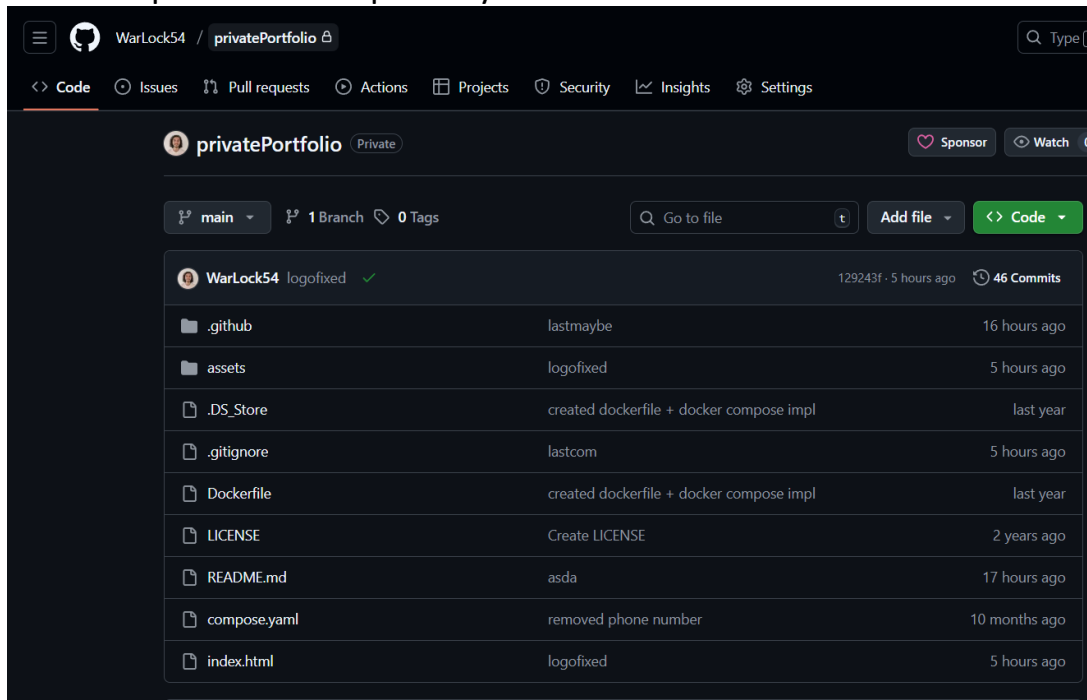


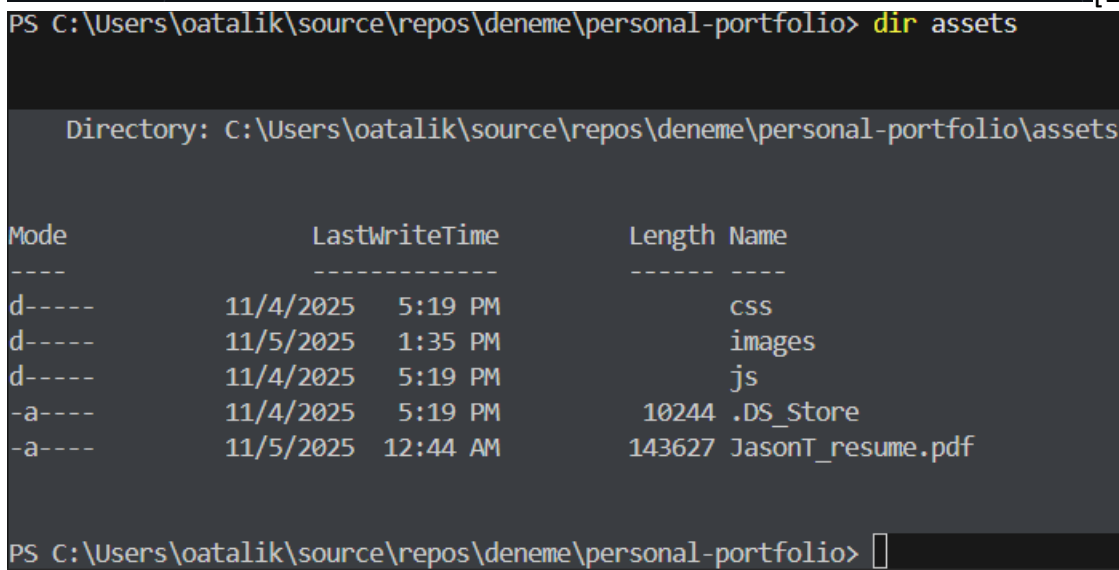
Static Website Deployment with AWS S3 and CodePipeline (CI/CD)

This guide, as part of the AWS Fundamentals task, explains how to statically host a simple personal portfolio site located in a GitHub repository on AWS S3 and automatically deploy it (CI/CD) using AWS CodePipeline. Goal: To automatically update the live website in the S3 bucket with every push made to the main branch on GitHub.

Step 1: GitHub Repository



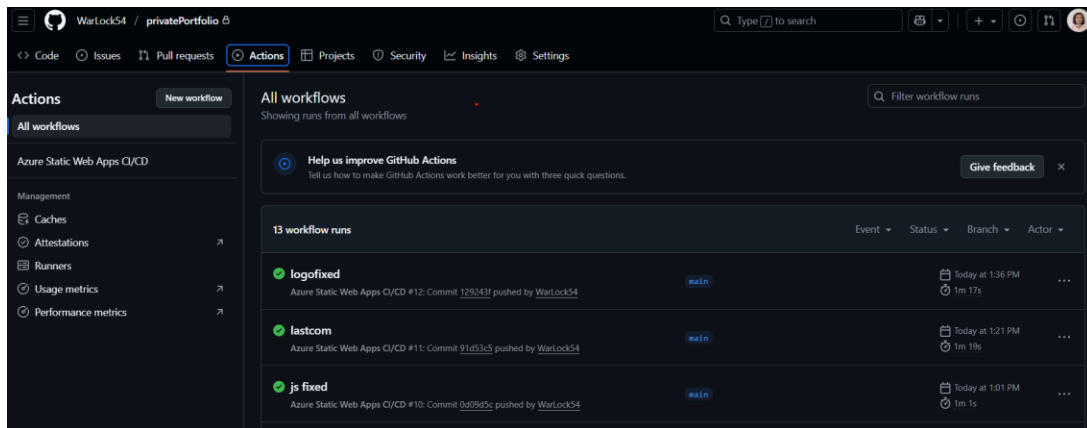
[1.1 Image]



[1.2

Image]

The images you see above are attached to the github private repository and file paths.



[1.3 Image]

The actions that work when I push to my repository on GitHub are attached.

Step 2: Creating and Configuring an AWS S3 Bucket

In this step, we will configure the S3 bucket that will host our site and make it publicly accessible on the internet.

2.1 Creating a Bucket:

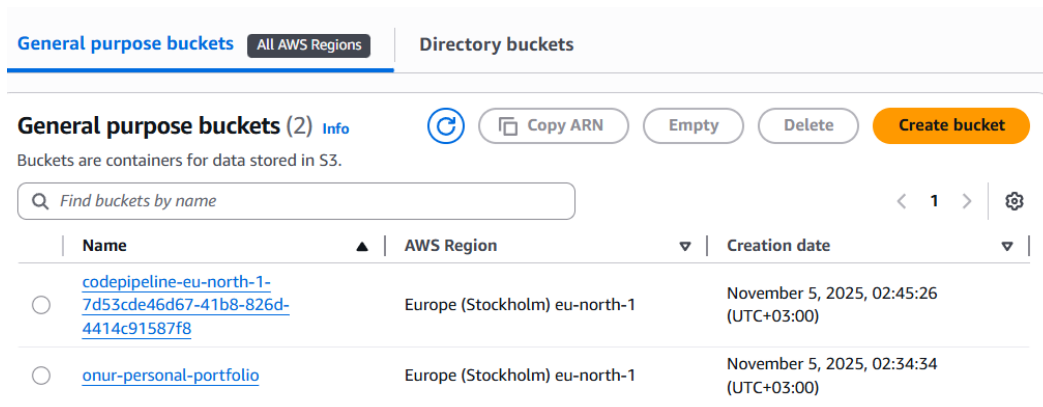
Go to the AWS S3 console and click “Create bucket.”

Choose a unique name for the bucket (e.g., onur-personal-portfolio).

Select your Region.

Uncheck the “Block all public access” option and check the confirmation box.

Create the bucket.



[2.1 Image]

2.2 Enabling Static Website Hosting:

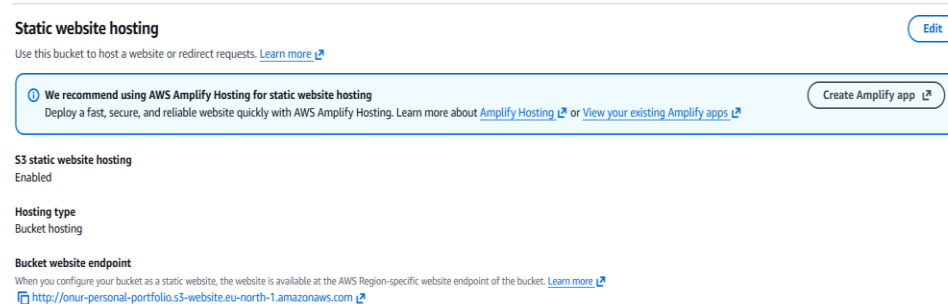
Go into the bucket you created and open the “Properties” tab.

Scroll down to find the “Static website hosting” section and click “Edit.”

Check the “Enable” option.

Type index.html in the “Index document” field.

Save your changes. This will give you a Bucket website endpoint URL (e.g., <http://onur-personal-portfolio.s3-website.eu-north-1.amazonaws.com>).



[2.2 Image]

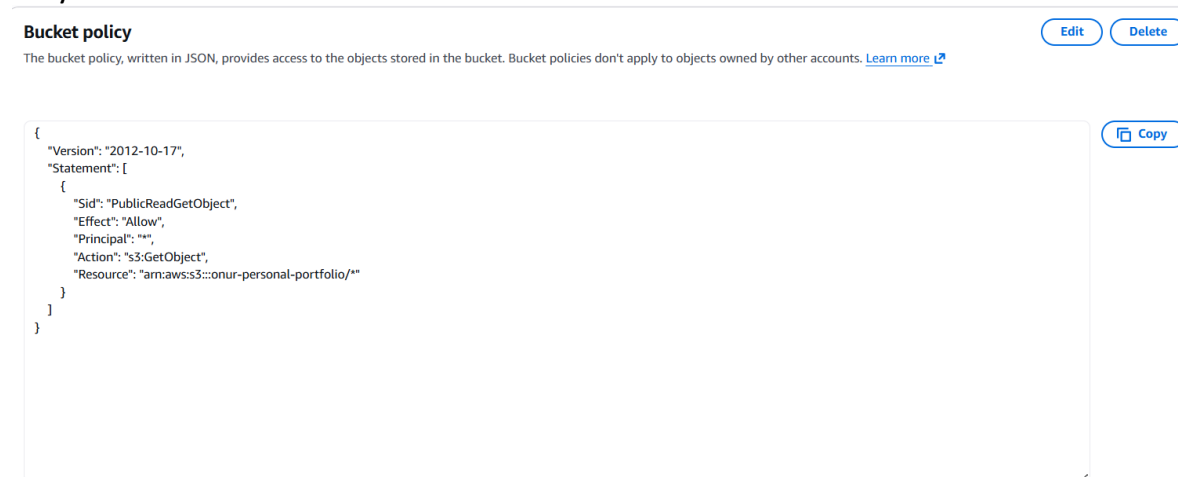
2.3 Adding a Bucket Policy for Public Access:

Go to the “Permissions” tab for your Bucket.

Find the “Bucket policy” section and click “Edit.”

Paste the following JSON policy. IMPORTANT: Replace the onur-personal-portfolio part in the Resource section with your own bucket name.

This policy allows anyone on the internet (Principal: “*”) to read (s3:GetObject) the files on your site.



[2.3

Image]

Step 3: Automated Deployment with AWS CodePipeline (CI/CD)

In this step, we will set up the automation pipeline that connects GitHub to S3.

Creating a New Pipeline:

Go to the AWS CodePipeline service and click “Create pipeline.”

Give your pipeline a name (e.g., personal-portfolio-deploy).

Select the “New service role” option (it will automatically generate the role name)

3.1 Source Stage:

Select GitHub (Version 2) as the “Source provider.”

Click “Create a connection” to connect to your GitHub account and authorize it.

Once the connection is established, select your personal-portfolio repository from the “Repository name” list.

Select main (or the relevant branch) as the “Branch name”.

Edit action

Action name

Choose a name for your action

Source

No more than 100 characters

Action provider

GitHub (via OAuth app)

Grant AWS CodePipeline access to your GitHub repository. This allows AWS CodePipeline to upload commits from GitHub to your pipeline.

[Connect to GitHub](#)

Repository WarLock54/privatePortfolio

Branch main

The GitHub (via OAuth app) action is not recommended

The selected action uses OAuth apps to access your GitHub repository. This is no longer the recommended method. Instead, choose the GitHub (via GitHub App) action to access your repository. GitHub Apps to manage authentication and can be shared with other resources. [Learn more](#)

Variable namespace - optional

Choose a namespace for the output variables from this action. You must choose a namespace if you want to use the variables this action produces in your configuration. [Learn more](#)

SourceVariables

Output artifacts

Choose a name for the output of this action.

SourceArtifact

No more than 100 characters

[3.1 Image]

3.2 Build Stage:

Since our project consists of static HTML/CSS/JS files and does not require a build process, we will skip this step.

Note: Here, because I selected a build provider and entered an invalid command like “honor,” the sequential output status fails with 127 (command not found). This step is unnecessary for static sites. Therefore, I removed it.

3.3 Deployment Phase:

Select Amazon S3 as the “Deploy provider”.

Select the region where you created your bucket as the Region.

Select your onur-personal-portfolio bucket from the “Bucket” list.

Check the “Extract file before deploy” option. This ensures the .zip file from GitHub is extracted to S3.

Edit action

Action name

Choose a name for your action

deploy

No more than 100 characters

Action provider

Amazon S3

Region

Europe (Stockholm)

Input artifacts

Choose an input artifact for this action. [Learn more](#)

SourceArtifact
Defined by: Source

No more than 100 characters

Bucket

onur-personal-portfolio

Deployment path - optional

☒ Extract file before deploy

The deployed artifact will be unzipped before deployment.

► Additional configuration

[3.3

Image]

Step 4: Fixing CodePipeline IAM Role Permissions (Important)

After your pipeline is created, it will most likely fail at the “Deploy” stage with an `s3:PutObject ... AccessDenied` error. This is because CodePipeline does not have permission to write files to your S3 bucket.

4.1 Finding the Service Role:

Find the name of the service role used by the pipeline from your CodePipeline settings or the error details in the “Deploy” stage

(`service-role/AWSCodePipelineServiceRole-eu-north-1-onur-personal-portfolio`).

4.2 Adding Permissions to the IAM Role:

Go to the AWS IAM service and open the “Roles” section.

Find and click on your CodePipeline's service role (e.g., `AWSCodePipelineServiceRole...`).

In the “Permissions” tab, select “Add permissions” -> “Create inline policy”.

Switch to the JSON tab and paste the following policy. Make sure to replace the `onur-personal-portfolio` parts with your own bucket name.

This policy grants CodePipeline permission to list your bucket (`ListBucket`) and upload objects/files to it (`PutObject`, `PutObjectAcl`).

```

JSON

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:PutObjectAcl"
      ],
      "Resource": "arn:aws:s3:::onur-personal-portfolio/*"
    },
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::onur-personal-portfolio"
    }
  ]
}

```

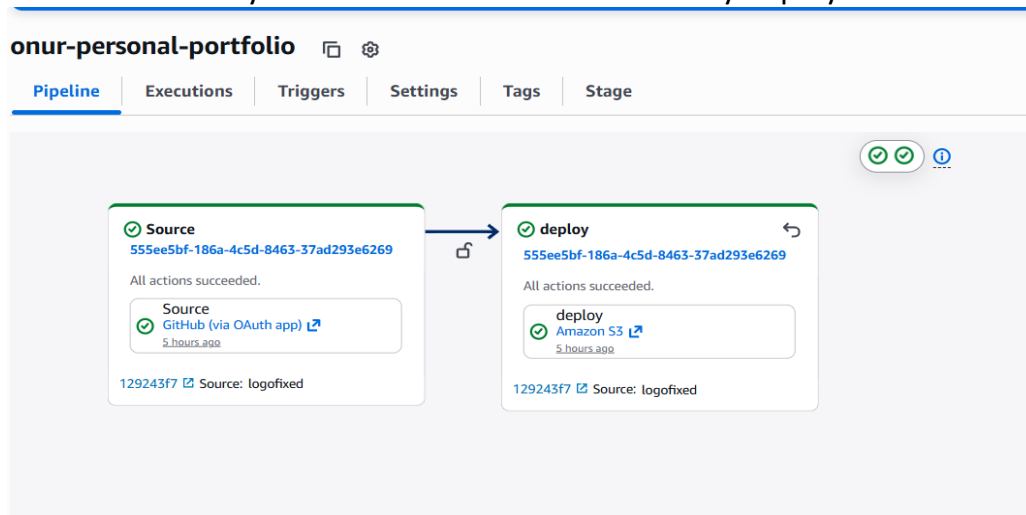
[4.2 Image]

Step 5: Results and Verification

After adding the permissions, return to your CodePipeline page and click the “Retry” button for the failed “Deploy” stage.

Verify that the Pipeline is now “Succeeded”.

Open the Bucket website endpoint URL you obtained in Step 2.2 (e.g., <http://onur-personal-portfolio.s3-website.eu-north-1.amazonaws.com>) in your browser. You will see that your website has been successfully deployed.



Test: Make a small change to the index.html file in your GitHub repository and push it. Within a few minutes, verify that CodePipeline automatically detected the change and updated your site.

This is the link=><http://onur-personal-portfolio.s3-website.eu-north-1.amazonaws.com/>

